
BEACONHOUSE NATIONAL UNIVERSITY

2nd semester OOP Project

Medical Store Management System

With Clinic Functionality

Object-Oriented Programming C++

Developed by:

Javeria Hussain

Student ID: F2025-0990

Table of Contents

1. Problem Statement	
1.1 Identified Challenges	
2. Scope Definition	
2.1 Project Objectives	
2.2 Features Included	
2.3 Features Not Included	
2.4 Target Users	
3. System Requirements	
3.1 Hardware Requirements	
3.2 Software Requirements	
3.3 Technical Dependencies	
4. System Architecture	
4.1 Class Structure	
4.2 OOP Concepts Applied	
4.3 File Structure	
5. System Workflow	
5.1 Program Initialization	
5.2 Admin Workflow	
5.3 Customer Workflow	
5.4 Program Exit	
6. Features Explanation	
6.1 Add Medicine (Admin Only)	
6.2 View All Stock (Admin Only)	
6.3 View Total Revenue (Admin Only)	
6.4 View Appointments (Admin Only)	
6.5 Search and Buy Medicine (Customer)	
6.6 Book Doctor Appointment (Customer)	
6.7 Input Validation & Error Handling	
7. User Manual	
7.1 Compilation	
7.2 Execution	
7.3 Admin Access	
7.4 Customer Access	
7.5 Common Issues & Solutions	
8. Technical Implementation	
8.1 Dynamic Memory Management	
8.2 File I/O Strategy	
8.3 Search Algorithm	
8.4 Validation Mechanisms	
9. Conclusion	
9.1 Key Achievements	

9.2 Learning Outcomes.....

10.GitHub Repo

1. Problem Statement

Managing a medical store involves handling a large volume of medicines, customer transactions, and clinic appointments on a daily basis. Manual management of these operations is prone to errors and inefficiencies.

1.1 Identified Challenges

- **Stock Tracking** — Monitoring medicine quantities manually is unreliable and time-consuming
- **Sales Records** — Recording each transaction by hand increases the risk of errors
- **Revenue Calculation** — Manual tallying of daily earnings is slow and inaccurate
- **Appointment Management** — Handling patient appointments without a system leads to scheduling conflicts
- **Data Integrity** — Paper-based records are vulnerable to loss, damage, and difficult retrieval

These challenges highlight the need for an automated software solution capable of managing store operations accurately, efficiently, and with persistent data storage.

2. Scope Definition

2.1 Project Objectives

- Manage medicine inventory with name, price, and stock quantity
- Handle customer purchases with automatic stock updates
- Calculate and display total revenue earned by the store
- Enable patients to book appointments with a doctor
- Save medicine data persistently using file handling
- Separate Admin and Customer access with role-based permissions

2.2 Features Included

Feature	Description
Medicine Management	Add, view, and search medicines with stock tracking
Sales Processing	Purchase medicines with automatic quantity reduction
Revenue Tracking	Real-time calculation of total earnings and units sold
Appointment Booking	Book doctor appointments with patient details and time validation
File Persistence	All medicine data saved to medicines.txt in pipe-delimited format

Input Validation	24-hour time format validation and integer input protection
Dynamic Memory	Arrays grow automatically when capacity is reached
Role-Based Access	Admin login with password; customer access without authentication

2.3 Features Not Included

- No graphical user interface (terminal-based only)
- No database integration (file-based storage only)
- No online payment processing
- No multi-user concurrent access
- Appointments are session-based (not saved to file)

2.4 Target Users

Role	Capabilities
Admin (Store Owner)	Add medicines, view stock, check revenue, view appointments
Customer	Search and buy medicines, book doctor appointments

3. System Requirements

3.1 Hardware Requirements

- Processor: Any modern CPU (Intel/AMD)
- RAM: Minimum 512 MB
- Storage: 10 MB free disk space

3.2 Software Requirements

- Operating System: Windows , Linux, or macOS
- C++ Compiler: GCC (g++) 7.0+ or any C++11 compatible compiler
- Terminal: Command Prompt, PowerShell, or any UNIX terminal

3.3 Technical Dependencies

- Standard C++ Libraries: iostream, fstream, string, stdexcept
 - File System: Read/write permissions in program directory
 - ANSI Color Support: Terminal must support ANSI escape codes (for colors)
4. System Architecture

4.1 Class Structure

The system is built using 7 classes following object-oriented design principles:

Class	Type	Purpose
Person	Abstract Base	Parent class for Patient, Doctor, Admin (defines pure virtual display)
Patient	Child	Inherits from Person, adds age attribute
Doctor	Child	Inherits from Person, adds specialization
Admin	Child	Inherits from Person, adds password authentication
Medicine	Standalone	Stores medicine details with static sales counter
Appointment	Standalone	Associates Patient with Doctor for appointments

MedicalStore	Controller	Main class managing arrays, file I/O, and menus
--------------	------------	---

4.2 OOP Concepts Applied

Concept	Implementation
Classes & Objects	All 7 classes with objects created at runtime
Encapsulation	All data members private/protected with public getters/setters
Inheritance	Patient, Doctor, Admin extend Person
Abstraction	Person is abstract class with pure virtual display()
Pure Virtual Function	virtual void display() = 0 in Person class
Function Overriding	Each child class overrides display()
Association	Appointment links Patient and Doctor objects
File Handling	saveToFile() and loadFromFile() methods
Dynamic Arrays	new/delete with automatic growth using growMedicinesArray()
Static Keyword	totalMedicinesSold shared across all Medicine objects
this Pointer	Used in all constructors and setter methods
Exception Handling	try-catch blocks in loadFromFile() for corrupted data

4.3 File Structure

The system uses a single text file (medicines.txt) for persistent storage:

Format: Each medicine stored on one line with fields separated by |

```
3
PANADOL|15.5|98
BRUFEN|25.0|47
DISPRIN|10.0|200
```

- Line 1: Total count of medicines
- Remaining lines: NAME|PRICE|QUANTITY format

5. System Workflow

5.1 Program Initialization

1. Program starts — MedicalStore object created in main()
2. Constructor allocates dynamic arrays for medicines and appointments
3. loadFromFile() reads medicines.txt and populates medicines array
4. Main menu displays with three options

```
=====
WELCOME TO MEDICAL STORE SYSTEM
Initializing system...
=====
[Data loaded from file successfully!]
=====
MEDICAL STORE MANAGEMENT SYSTEM
=====
1. Login as Admin (Store Owner)
2. Continue as Customer
3. Exit Program
=====
Enter your choice: 
```

5.2 Admin Workflow

5. Admin selects option 1 from main menu
6. System prompts for password (default: admin123)
7. On successful authentication, admin menu displays
8. Admin can: Add medicines, View stock, Check revenue, View appointments
9. All changes are immediately saved to medicines.txt


```
=====
Enter your choice: 1
```

```
Enter Admin Password: admin123
Access Granted! Welcome, Admin.
```

```
=====
ADMIN MENU (Store Owner)
```

- ```
=====
1. Add New Medicine
2. View All Stock
3. View Total Revenue
4. View All Appointments
5. Back to Main Menu
=====
```

```
=====
Enter your choice: 1
```

```
:::::::::ADD NEW MEDICINE:::::::::
```

```
Enter Medicine Name : PANADOL
Enter Price (Rs.) : 50
Enter Quantity in Stock : 100
Medicine added successfully!
```

```
=====
Enter your choice: 2
```

```
:::::::::ALL MEDICINES IN STOCK:::::::::
```

```
Medicine #1:
MedicineName : PANADOL
Price : Rs./ 50
In Stock : 100 units

```

```
[Total Units Sold So Far: 0]
```

```
=====
Enter your choice: 3
```

```
:::::::::TOTAL REVENUE:::::::::
```

```
Total Earnings : Rs./ 0
Total Units Sold : 0
```

```
=====
Enter your choice: 4
```

```
:::::::::ALL APPOINTMENTS:::::::::
```

```
No appointments booked yet.
```

```
=====
Enter your choice: 5
```

```
Going back to Main Menu...
```

```
=====
MEDICAL STORE MANAGEMENT SYSTEM
```

- ```
=====
1. Login as Admin (Store Owner)
2. Continue as Customer
3. Exit Program
=====
```

```
Enter your choice: 
```

5.3 Customer Workflow

10. Customer selects option 2 from main menu (no password required)
11. Customer menu displays with two options
12. Search and Buy: Enter medicine name → System searches (case-insensitive) → Display details
→ Enter quantity → Process sale → Update stock and revenue
13. Book Appointment: Enter patient details → Validate age → Validate 24-hour time format →
Confirm booking

```
:::::::::Welcome, Customer!:::::::::
=====
                CUSTOMER MENU
=====
    1. Search and Buy Medicine
    2. Book Doctor Appointment
    3. Back to Main Menu
=====
Enter your choice: █
```

```
:::::::::BUY MEDICINE:::::::::
Enter Medicine Name (IN CAPITALS) : panadol

Medicine Found:
MedicineName : PANADOL
Price       : Rs./ 50
In Stock    : 100 units
Enter Quantity to Buy : 50

:::::::::Purchase Successful!:::::::::
Medicine  : PANADOL
Quantity  : 50 units
Total Bill: Rs. 2500
```

```
.....:BOOK DOCTOR APPOINTMENT:.....:
Enter Your Name      : Fatima Zahra
Enter Contact Number : 0300000000
Enter Your Age       : 20
Enter Preferred Time (24-hr format HH:MM, e.g. 14:30) : 12:00

Appointment Booked Successfully!
Doctor: Dr.Javeria Hussain (General Physician)
Time   : 12:00
```

5.4 Program Exit

14. User selects Exit from main menu
15. Destructor automatically called when MedicalStore object goes out of scope
16. saveToFile() writes all updated medicine data to medicines.txt
17. delete[] frees dynamically allocated arrays
18. Program terminates gracefully

6. Features Explanation

6.1 Add Medicine (Admin Only)

Admin can add new medicines to the inventory with automatic uppercase conversion and validation.

- Input: Medicine name, price, quantity
- Validation: Price and quantity cannot be negative
- Behavior: Name converted to uppercase for consistent search
- Storage: Array grows automatically if capacity reached
- Persistence: Immediately saved to medicines.txt

```

=====
                ADMIN MENU (Store Owner)
=====
1. Add New Medicine
2. View All Stock
3. View Total Revenue
4. View All Appointments
5. Back to Main Menu
=====
Enter your choice: 1

:::::::::ADD NEW MEDICINE:::::::::
Enter Medicine Name      : SEPTRAN SYRUP
Enter Price (Rs.)        : 100
Enter Quantity in Stock  : 20
Medicine added successfully!

```

6.2 View All Stock (Admin Only)

Displays complete inventory with total units sold across all medicines.

- Shows: Medicine number, name, price, current stock
- Displays static counter of total units sold

```

:::::::::ALL MEDICINES IN STOCK:::::::::

Medicine #1:
MedicineName : PANADOL
Price        : Rs./ 50
In Stock     : 50 units
-----

Medicine #2:
MedicineName : SEPTRAN SYRUP
Price        : Rs./ 100
In Stock     : 20 units
-----

[Total Units Sold So Far: 0]

```

6.3 View Total Revenue (Admin Only)

Real-time financial summary showing total earnings and units sold.

- Total Earnings: Sum of all sale amounts in PKR
- Total Units Sold: Static counter shared across all Medicine objects
- Updates: Automatically after every purchase

```
=====
                ADMIN MENU (Store Owner)
=====
1. Add New Medicine
2. View All Stock
3. View Total Revenue
4. View All Appointments
5. Back to Main Menu
=====
Enter your choice: 3

:::::::::TOTAL REVENUE:::::::::
Total Earnings : Rs./ 2500
Total Units Sold : 50
```

6.4 View Appointments (Admin Only)

Lists all booked appointments with patient and doctor details.

- Session-based: Appointments stored in memory only (not saved to file)
- Displays: Patient name, contact, age, appointment time, doctor details
- Association: Each appointment links Patient and Doctor objects

```
:::::::::ALL APPOINTMENTS:::::::::
```

Appointment #1:

```
Patient Name : Fatima Zahra
Contact No.  : 0300000000
Age          : 20
Time Booked  : 12:00
Doctor Name  : Dr.Javeria Hussain
Field        : General Physician
-----
```

6.5 Search and Buy Medicine (Customer)

Customer searches by name and purchases medicines with automatic stock updates.

- Input: Medicine name (case-insensitive via toUpper() conversion)
- Search: Linear search through medicines array
- Validation: Checks stock availability and quantity limits
- Processing: Reduces stock, updates revenue, increments static sales counter
- Error Handling: Clear messages for not found, out of stock, insufficient quantity

```
:::::::::Welcome, Customer!:::::::::
```

```
=====
```

CUSTOMER MENU

```
=====
```

1. Search and Buy Medicine
2. Book Doctor Appointment
3. Back to Main Menu

```
=====
```

Enter your choice: 1

```
:::::::::BUY MEDICINE:::::::::
```

```
Enter Medicine Name (IN CAPITALS) : detran p
Medicine 'DETRAN P' not found in store.
```

```
:::::::::BUY MEDICINE:::::::::
Enter Medicine Name (IN CAPITALS) : septran syrup

Medicine Found:
MedicineName : SEPTRAN SYRUP
Price       : Rs./ 100
In Stock    : 20 units
Enter Quantity to Buy : 2

:::::::::Purchase Successful!:::::::::
Medicine  : SEPTRAN SYRUP
Quantity  : 2 units
Total Bill: Rs. 200
```

6.6 Book Doctor Appointment (Customer)

Allows patients to schedule appointments with built-in input validation.

- Input: Patient name, contact number, age, preferred time
- Time Validation: Must be in 24-hour HH:MM format (00:00 to 23:59)
- Validation Loop: Keeps asking until valid time entered
- Default Doctor: Dr. Javeria Hussain (General Physician)
- Storage: Appointments array (session-based, not persisted)

```
::::::::::BOOK DOCTOR APPOINTMENT::::::::::  
Enter Your Name      : Aryan  
Enter Contact Number : 0340000000  
Enter Your Age       : 15  
Enter Preferred Time (24-hr format HH:MM, e.g. 14:30) : 10:20  
  
Appointment Booked Successfully!  
Doctor: Dr.Javeria Hussain (General Physician)  
Time   : 10:20
```

6.7 Input Validation & Error Handling

Robust validation prevents crashes and ensures data integrity.

- Integer Input: getIntInput() function loops until valid number entered
- Price Input: Inline loop handles non-numeric input
- Time Format: isValidTime() enforces HH:MM with range checks
- File Corruption: try-catch blocks in loadFromFile() skip corrupted lines
- Negative Values: Prevented for price and quantity

```
::::::::::BOOK DOCTOR APPOINTMENT::::::::::  
Enter Your Name      : Ahmed  
Enter Contact Number : 03000000  
Enter Your Age       : 34  
Enter Preferred Time (24-hr format HH:MM, e.g. 14:30) : 45:233  
ERROR: Invalid time! Use HH:MM format (00:00 to 23:59).  
  
Enter Preferred Time (24-hr format HH:MM, e.g. 14:30) : 
```


7. User Manual

7.1 Compilation

Open terminal in the directory containing `medical_store.cpp` and run:

```
g++ medical_store.cpp -o medical_store
```

7.2 Execution

Run the compiled program:

Windows: `medical_store.exe`

Linux/Mac: `./medical_store`

7.3 Admin Access

19. Select option 1 from main menu
20. Enter password: `admin123`
21. Access granted — admin menu appears
22. Perform operations and select option 5 to return to main menu

7.4 Customer Access

23. Select option 2 from main menu (no password required)
24. Choose Search and Buy or Book Appointment
25. Follow on-screen prompts
26. Select option 3 to return to main menu

7.5 Common Issues & Solutions

Issue	Solution
Program crashes on startup	Delete <code>medicines.txt</code> file (old format incompatible)
Colors not displaying	Use a terminal supporting ANSI escape codes
Compilation error: 'std' not found	Compile with C++11 flag: <code>g++ -std=c++11 medical_store.cpp</code>
File not saving	Ensure write permissions in program directory
Invalid input causes infinite loop	This is fixed — <code>getIntInput()</code> validates all integer inputs

8. Technical Implementation

8.1 Dynamic Memory Management

Arrays allocated on heap with automatic growth:

- Initial capacity: 10 medicines, 10 appointments
- Allocation: `new Medicine[capacity]` in constructor
- Growth: `growMedicinesArray()` doubles capacity when full
- Cleanup: `delete[]` in destructor prevents memory leaks

8.2 File I/O Strategy

Single-line pipe-delimited format for efficient parsing:

- Save: Loop through array writing `NAME|PRICE|QUANTITY` per line
- Load: Read count, then parse each line using `find()` and `substr()`
- Exception Handling: try-catch around `stod()/stoi()` for corrupted data
- Timing: Load on startup, save after every add/purchase and on exit

8.3 Search Algorithm

Linear search with case-insensitive matching:

- Input normalized: `toUpper()` converts user input to uppercase
- Storage normalized: All medicine names stored in uppercase
- Comparison: Simple string equality check in loop
- Performance: $O(n)$ complexity acceptable for typical store inventory

8.4 Validation Mechanisms

Input Type	Validation Method
Menu Choice	<code>getIntInput()</code> loops until valid integer
Medicine Price	Inline <code>while(cin.fail())</code> loop
Quantity	<code>getIntInput()</code> + range check (must be > 0)
Appointment Time	<code>isValidTime()</code> enforces HH:MM format with range validation
File Data	try-catch blocks handle exceptions from <code>stod()/stoi()</code>

9. Conclusion

The Medical Store Management System successfully demonstrates the practical application of object-oriented programming principles in solving a real-world business problem. The project achieves all stated objectives through a clean architecture that separates concerns, validates inputs, and persists data reliably.

9.1 Key Achievements

- Complete implementation of 12 OOP concepts including abstraction and exception handling
- Robust input validation preventing all common crash scenarios
- Efficient file-based persistence with graceful corruption handling
- Clean separation of admin and customer functionality
- Professional terminal-based interface with color-coded feedback

9.2 Learning Outcomes

- Deep understanding of inheritance hierarchies and polymorphism
- Practical experience with dynamic memory management and avoiding leaks
- Mastery of file I/O with structured data formats
- Implementation of real-world validation and error handling
- Application of static members for shared state tracking

GitHub Repo:

<https://github.com/itx-javeria/Medical-Store-Management-System>

Visit to see the source code of this project.